Trabajo Práctico Nº 1

Conceptos básicos para la construcción de Sistemas Distribuidos

Fecha de entrega: 20/03/26

Integrantes:

Tobias Avila

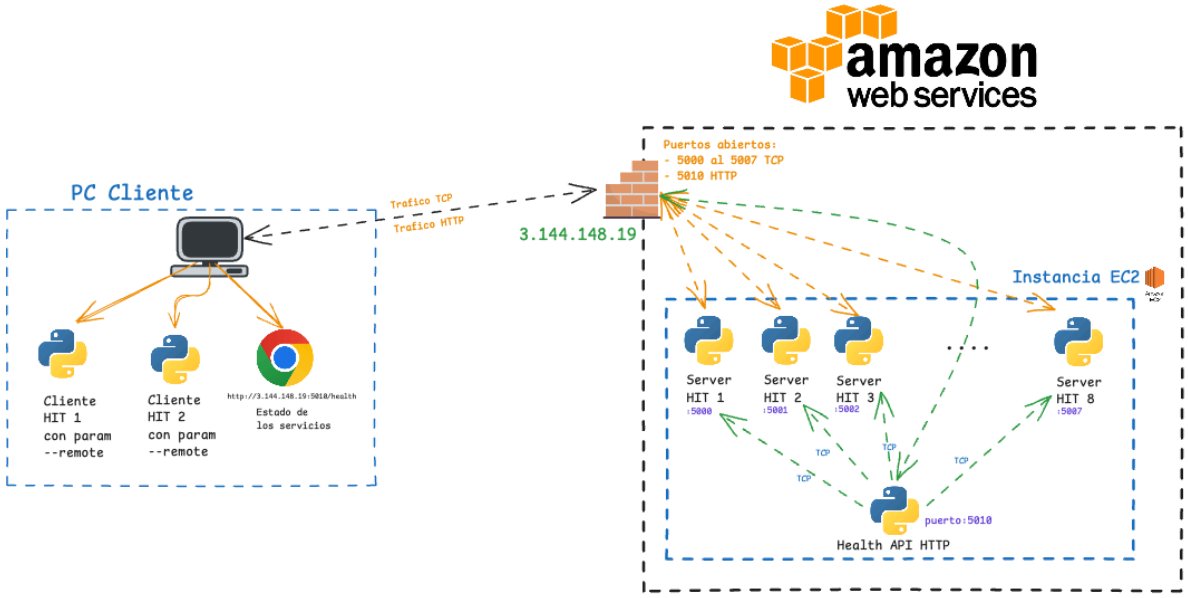
Renzo Robles

Axel Hoffmann

Introducción

El presente informe detalla el desarrollo y la implementación del Trabajo Práctico Nº 1, enfocado en los conceptos básicos para la construcción de Sistemas Distribuidos. A lo largo de este documento, se expone la evolución progresiva de una arquitectura de red, comenzando desde una comunicación cliente-servidor fundamental utilizando sockets TCP , hasta alcanzar una topología descentralizada estructurada en nodos concurrentes. Asimismo, se documenta la transición en los mecanismos de serialización y comunicación, migrando de mensajes en formato JSON hacia la implementación de servicios gRPC con Protocol Buffers, lo cual permitió optimizar significativamente el intercambio de datos. Finalmente, se analizan las decisiones de diseño adoptadas para garantizar la tolerancia a fallos y la sincronización de estado, evaluando el comportamiento del sistema desplegado en un entorno real sobre instancias de Amazon EC2.

Diagrama de arquitectura



Hit #1

Elabore un código de servidor TCP para B que espere el saludo de A y lo responda. Elabore un código de cliente TCP para A que se conecte con B y lo salude. Implementación de comunicación TCP entre dos nodos:

- **Nodo B (server_b.py)**: servidor TCP que espera el saludo de A y responde.
- **Nodo A (client_a.py)**: cliente TCP que se conecta a B y lo salude.

Cómo ejecutar: Requiere Python 3.x (sin dependencias externas, solo stdlib).

1. Iniciar el servidor B (en una terminal)

python3 tp1/HIT1/server_b.py

Salida esperada:

[B] Servidor escuchando en 0.0.0.0:9000

2. Iniciar el cliente A (en otra terminal)

python3 tp1/HIT1/client_a.py

Salida esperada del cliente:

[A] Conectando a 127.0.0.1:9000... [A] Conexion establecida.
[A] Saludo enviado: Hola B, soy A!
[A] Respuesta de B: Hola A, soy B. Saludo recibido!
[A] Conexion cerrada.

Salida esperada del servidor:

[B] Conexion aceptada desde ('127.0.0.1', <puerto_efimero>)
[B] Recibi: Hola B, soy A!
[B] Respuesta enviada: Hola A, soy B. Saludo recibido!
[B] Conexion cerrada.

Deploy en EC2

El servidor corre como servicio systemd en EC2 en el puerto TCP 5000.

```
# Verificar que el servicio está activo
ssh -i clave-grupo404.pem ubuntu@3.144.148.19 "systemctl status hit1-server"

# Ver logs en tiempo real
ssh -i clave-grupo404.pem ubuntu@3.144.148.19 "journalctl -fu hit1-server"
```

Flags del cliente:

- **(ninguno)** | Host: 127.0.0.1 | Puerto: 9000 | Testing local / retrocompatible
- **--local** | Host: 127.0.0.1 | Puerto: 5000 | Local con el mismo puerto que EC2
- **--remote** | Host: 3.144.148.19 | Puerto: 5000 | Contra el servidor en EC2

```
# Probar contra EC2 (servidor ya corriendo)
python3 tp1/HIT1/client_a.py --remote
```

```
# Probar local con mismo puerto que EC2
PORT=5000 python3 tp1/HIT1/server_b.py &
python3 tp1/HIT1/client_a.py --local
```

Hit #2

Extensión del HIT #1: el nodo A ahora implementa reconexión automática. Si B cierra la conexión o es terminado abruptamente, A detecta el error y reintenta conectarse cada RECONNECT_DELAY segundos hasta restablecer la comunicación. El nodo B se mantiene igual en esencia, pero ahora acepta múltiples conexiones en un loop para poder recibir los reintentos de A.

Decisiones de Diseño:

- **Loop infinito en A con time.sleep**: la reconexión se implementa con un delay fijo entre intentos (RECONNECT_DELAY = 3s).
- **Captura de excepciones de red**: se capturan ConnectionRefusedError, ConnectionResetError y OSError.
- **Nuevo socket por intento**: se crea un socket nuevo en cada iteración del loop en lugar de reusar el anterior.

Cómo ejecutar: Requiere Python 3.x (sin dependencias externas, solo stdlib).

1. Iniciar el servidor B

```
python3 tp1/HIT2/server_b.py
```

2. Iniciar el cliente A (en otra terminal)

```
python3 tp1/HIT2/client_a.py
```

A se conecta, saluda y vuelve a intentarlo cada 3 segundos.

3. Probar la reconexión Con ambos corriendo, matá B con Ctrl+C. A detectará el error y mostrará:

```
[A] Error de conexion: [Errno 61] Connection refused
[A] Reintentando en 3 segundos...
```

Deploy en EC2 El servidor corre como servicio systemd en EC2 en el puerto TCP 5001.

Ver logs en tiempo real
ssh -i clave-grupo404.pem ubuntu@3.144.148.19 "journalctl -fu hit2-server"

Flags del cliente:

- **(ninguno)** | Host: 127.0.0.1 | Puerto: 9000 | Testing local / retrocompatible
- **--local** | Host: 127.0.0.1 | Puerto: 5001 | Local con el mismo puerto que EC2
- **--remote** | Host: 3.144.148.19 | Puerto: 5001 | Contra el servidor en EC2

Probar reconexión contra EC2: matar el servicio desde otra terminal para simular caída

python3 tp1/HIT2/client_a.py --remote

Hit #3

Extensión del HIT #2: ahora el foco está en el nodo B. Si A se desconecta abruptamente (proceso matado, crash, corte de red), B captura la excepción y continúa escuchando nuevas conexiones sin necesidad de reiniciarse. El nodo A se mantiene conectado en un loop luego del saludo, permitiendo simular una desconexión abrupta matando su proceso.

Decisiones de Diseño:

- **listen(5)**: se aumenta el backlog a 5 para que el SO pueda encolar conexiones entrantes mientras B procesa la actual.
- **BrokenPipeError y ConnectionResetError**: cubren los dos escenarios principales de desconexión abrupta.

Cómo ejecutar: Requiere Python 3.x (sin dependencias externas, solo stdlib).

1. Iniciar el servidor B

python3 tp1/HIT3/server_b.py

2. Iniciar el cliente A (en otra terminal)

python3 tp1/HIT3/client_a.py

3. Probar la robustez de B Matá el proceso A con Ctrl+C. B mostrará:

[B] A se desconecto abruptamente: ...
[B] Conexion con A cerrada. Esperando nueva conexion...

Deploy en EC2 El servidor corre como servicio systemd en EC2 en el puerto TCP 5002.

```
# Ver logs en tiempo real
ssh -i clave-grupo404.pem ubuntu@3.144.148.19 "journalctl -fu hit3-server"
```

Flags del cliente:

- **(ninguno)** | Host: 127.0.0.1 | Puerto: 9000 | Testing local / retrocompatible
- **--local** | Host: 127.0.0.1 | Puerto: 5002 | Local con el mismo puerto que EC2
- **--remote** | Host: 3.144.148.19 | Puerto: 5002 | Contra el servidor en EC2

```
# Conectar al servidor en EC2 y luego matar el cliente con Ctrl+C para probar robustez
python3 tp1/HIT3/client_a.py --remote
```

Hit #4

Refactorización de A y B en un único programa node_c.py que opera simultáneamente como cliente y servidor mediante dos threads:

- **Thread servidor**: escucha en su propio puerto y responde saludos entrantes.
- **Thread cliente**: se conecta al otro nodo C y le envía un saludo, con reintentos automáticos si el par aún no está disponible.

Decisiones de Diseño:

- **Un thread por rol**: el servidor y el cliente corren en threads separados para operar de forma verdaderamente concurrente. Usar daemon=True garantiza que los threads se terminan cuando el proceso principal recibe Ctrl+C.
- **Reintentos en el cliente**: se hereda la lógica del HIT #2.
- **Cliente termina tras el saludo**: el thread cliente hace su trabajo y finaliza. El servidor sigue corriendo para aceptar futuros saludos.

Cómo ejecutar: Requiere Python 3.x (sin dependencias externas, solo stdlib).

```
# Terminal 1 — C1
python3 tp1/HIT4/node_c.py --listen-port 9001 --remote-host 127.0.0.1 --remote-port 9002

# Terminal 2 — C2
python3 tp1/HIT4/node_c.py --listen-port 9002 --remote-host 127.0.0.1 --remote-port 9001
```

Salida esperada (C1):

[C-SERVER] Escuchando en 0.0.0.0:9001
[C-CLIENT] Intento #1 conectando a 127.0.0.1:9002...
[C-CLIENT] Error: [Errno 61] Connection refused. Reintentando en 2s...
[C-CLIENT] Intento #2 conectando a 127.0.0.1:9002...
[C-CLIENT] Saludo enviado: Hola! Soy C en puerto 9001.
[C-SERVER] Recibi de ('127.0.0.1', ...): Hola! Soy C en puerto 9002. [C-CLIENT] Respuesta de 127.0.0.1:9002: Hola! Soy C en puerto 9002. Saludo recibido.
[C-SERVER] Respuesta enviada a ('127.0.0.1', ...)

Deploy en EC2 En EC2 corren dos instancias del nodo como servicios systemd:

- hit4-node-c1 | TCP 5003 | Conecta a localhost:5004
- hit4-node-c2 | TCP 5004 | Conecta a localhost:5003

Flags del cliente:

- **--local** | Efecto: --remote-host 127.0.0.1 --remote-port 5003
- **--remote** | Efecto: --remote-host 3.144.148.19 --remote-port 5003

Hit #5

Extensión del HIT #4: todos los mensajes entre nodos C se envían y reciben en formato JSON. Se agregan dos funciones de bajo nivel (send_json / recv_json) que encapsulan la serialización y el framing TCP.

Decisiones de Diseño:

- **Newline como delimitador de mensajes**: Terminar cada JSON con \n y leer hasta encontrar ese carácter resuelve el problema de forma simple sin añadir dependencias.
- **send_json / recv_json como helpers**: encapsular la serialización en dos funciones hace que el resto del código trabaje solo con dicts de Python.
- **Campo timestamp en UTC (ISO 8601)**: facilita correlacionar eventos entre nodos en distintas zonas horarias.
- **Campo type**: permite extender el protocolo en HITs futuros.

Cómo ejecutar: Requiere Python 3.x (sin dependencias externas, solo stdlib).

Terminal 1 — C1

python3 tp1/HIT5/node_c.py --listen-port 9001 --remote-host 127.0.0.1 --remote-port 9002

Terminal 2 — C2

python3 tp1/HIT5/node_c.py --listen-port 9002 --remote-host 127.0.0.1 --remote-port 9001

Salida esperada (C1):

[C-SERVER] Escuchando en 0.0.0.0:9001
[C-CLIENT] Intento #1 conectando a 127.0.0.1:9002...
[C-CLIENT] JSON enviado: {"type": "greeting", "from_port": 9001, ...}
[C-SERVER] JSON recibido de ('127.0.0.1', ...): {"type": "greeting", "from_port": 9002, ...}
[C-CLIENT] JSON recibido: {"type": "greeting_response", "from_port": 9002, ...}
[C-SERVER] JSON enviado a ('127.0.0.1', ...): {"type": "greeting_response", "from_port": 9001, ...}

Deploy en EC2

- hit5-node-c1 | TCP 5003 | Conecta a localhost:5004
- hit5-node-c2 | TCP 5004 | Conecta a localhost:5003

Flags del cliente:

- **--local** | Efecto: --remote-host 127.0.0.1 --remote-port 5004
- **--remote** | Efecto: --remote-host 3.144.148.19 --remote-port 5004

Hit #6

Introduce un nuevo tipo de nodo: D, que actúa como registro de contactos. Node D expone un servidor TCP (puerto 9000) donde los nodos C se registran, y un servidor HTTP (puerto 8080) con endpoints de estado y listado. Node C ya no necesita saber las IPs de sus pares. Al iniciarse, C elige un puerto aleatorio, se registra en D y recibe la lista de peers activos a los que les envía un saludo JSON.

Decisiones de Diseño:

- **Thread por conexión en D**: cada registro de C se maneja en su propio thread.
- **Puerto aleatorio en C**: C hace bind("0.0.0.0", 0) y lee el puerto asignado por el SO.
- **_registry_lock**: protege la lista compartida de accesos concurrentes entre threads de registro.

- **_get_own_ip()**: detecta la IP saliente de C conectándose (sin datos) a 8.8.8.8.

Cómo ejecutar:

```
# Terminal 1 — Node D
uvicorn tp1.HIT6.node_d:app --host 0.0.0.0 --port 8080

# Terminales 2, 3, 4 — instancias de Node C
python3 tp1/HIT6/node_c.py --local # conecta a 127.0.0.1:5005
```

Verificar el estado local: curl http://localhost:8080/health curl http://localhost:8080/nodes

Deploy en EC2 Node D corre como servicio systemd hit6-node-d.

- TCP registro (node_c conecta aquí): 5005
- HTTP FastAPI (interno): 8086

Flags del cliente:

- **--local** | Host: 127.0.0.1 | Puerto: 5005 | Node D corriendo localmente
- **--remote** | Host: 3.144.148.19 | Puerto: 5005 | Node D en EC2
- **--registry-host X --registry-port Y** | manual | manual | Cualquier otro destino

Hit #7

Extiende el HIT #6 con un sistema de ventanas de tiempo de 1 minuto coordinado por D. Regla central: cuando C se inscribe a las 11:28:34, queda anotado para la ventana de las 11:29. Solo cuando el reloj llega a las 11:29:00 esa inscripción se activa y C pasa a ser visible para los nuevos C que lleguen. Mientras tanto, C solo ve los peers de la ventana actual, no los de la próxima. D mantiene dos listas: current_window (nodos activos) y next_window (inscripciones pendientes). Cada 60 s (al cambio de minuto), la ventana que cierra se persiste en inscripciones.json.

Decisiones de Diseño:

- **_window_manager basado en polling cada 1 s**: compara el minuto actual con el anterior. Es más simple y robusto que calcular un sleep hasta el próximo minuto exacto.
- **Dos listas separadas + lock**: _current_window y _next_window comparten un único threading.Lock. La rotación es atómica dentro del bloque with _lock.

- **C recibe peers de current_window, no de next_window**: los nodos no saben a priori quiénes son sus pares para la próxima ventana.
- **Persistencia append-only**: se agrega una entrada por ventana cerrada, nunca se sobreescriben entradas anteriores.
- **INSCRIPCIONES_FILE configurable por env var**: permite cambiar la ruta del archivo sin modificar el código.

Endpoints HTTP de Node D:

- **GET /health**: Uptime, info de ventana actual y próxima.
- **GET /window/current**: Nodos activos en la ventana actual.
- **GET /window/next**: Nodos inscriptos para la próxima ventana.
- **GET /windows**: Historial completo desde inscripciones.json.
- **DELETE /windows**: Limpia estado en memoria y archivo (testing).

Cómo ejecutar:

```
# Terminal 1 — Node D

uvicorn tp1.HIT7.node_d:app --host 0.0.0.0 --port 8080

# C1 y C2 se inscriben (van a next_window)

python3 tp1/HIT7/node_c.py --local
python3 tp1/HIT7/node_c.py --local

# Esperar el cambio de minuto (~60s)

# C3 se inscribe y ve a C1/C2 como peers actuales

python3 tp1/HIT7/node_c.py --local
```

Verificar estado local:

```
curl http://localhost:8080/health
curl http://localhost:8080/window/current
curl http://localhost:8080/window/next
curl http://localhost:8080/windows
```

Deploy en EC2

Node D corre como servicio systemd hit7-node-d.

- TCP registro (node_c conecta aquí): 5006
- HTTP FastAPI (interno): 8087

Ver logs

ssh -i clave-grupo404.pem ubuntu@3.144.148.19 "journalctl -fu hit7-node-d"

Flags del cliente:

- **--local** | Host: 127.0.0.1 | Puerto: 5006 | Node D corriendo localmente
- **--remote** | Host: 3.144.148.19 | Puerto: 5006 | Node D en EC2
- **--registry-host X --registry-port Y** | manual | manual | Cualquier otro destino

Inscribir un nodo C contra EC2

python3 tp1/HIT7/node_c.py --remote

Hit #8

Refactorización del HIT #5 reemplazando toda la comunicación por gRPC con Protocol Buffers. Se definen dos servicios en sd2026.proto:

- **GreetingService** — entre nodos C (reemplaza el TCP directo del HIT #5)
- **RegistryService** — entre C y D (reemplaza el servidor TCP del HIT #6)

Comparación JSON (HIT #5) vs Protobuf (HIT #8):

- Tamaño del mensaje: JSON: 133 bytes | Protobuf: 68 bytes (~49% menos)
- Serialización/Deserialización: JSON: Manual | Protobuf: Generada
- Framing TCP: JSON: Newline manual | Protobuf: HTTP/2 (gRPC nativo)
- Type safety: JSON: Ninguna en runtime | Protobuf: Verificada por protoc

Decisiones de Diseño:

- **Puerto 0 en el servidor gRPC de C**: server.add_insecure_port("[::]:0") devuelve el puerto asignado por el SO.
- **sys.path.insert para los stubs**: Se agrega ese path al inicio para que los imports funcionen.
- **FastAPI se mantiene en D**: gRPC no reemplaza al endpoint HTTP público.

Cómo ejecutar:

0. Generar stubs (una sola vez)

pip install grpcio grpcio-tools
bash tp1/HIT8/generate_stubs.sh

1. Iniciar Node D

uvicorn tp1.HIT8.node_d:app --host 0.0.0.0 --port 8080

2. Iniciar instancias de Node C

python3 tp1/HIT8/node_c.py --local

3. Verificar curl

http://localhost:8080/health

Deploy en EC2 Node D corre como servicio systemd hit8-node-d.

- gRPC registro (node_c conecta aquí): 5007
- HTTP FastAPI (interno): 8088

Flags del cliente:

- **--local** | Host: 127.0.0.1 | Puerto: 5007 | Node D corriendo localmente
- **--remote** | Host: 3.144.148.19 | Puerto: 5007 | Node D en EC2
- **--registry-host X --registry-grpc-port Y** | manual | manual | Cualquier otro destino

Conclusiones finales

La realización de este trabajo práctico permitió llevar a la práctica los fundamentos teóricos de los sistemas distribuidos mediante una implementación técnica iterativa. El manejo manual de la red en los primeros estadios evidenció los desafíos inherentes a las comunicaciones distribuidas, exigiendo el diseño de mecanismos para la gestión de reconexiones automáticas y la tolerancia a desconexiones abruptas. Sin embargo, la posterior integración de tecnologías como gRPC y Protocol Buffers demostró una mejora sustancial en la eficiencia del sistema, reduciendo el tamaño de la carga útil respecto a JSON y aportando robustez mediante la validación de tipos. Por otro lado, la coordinación

de nodos mediante un sistema de ventanas de tiempo expuso la complejidad de administrar el estado distribuido y la concurrencia. En definitiva, el despliegue exitoso de la arquitectura como servicios en EC2 válida la resiliencia del diseño y proporciona una base sólida para el futuro desarrollo de APIs y sistemas escalables.